

Sapien: A Stateful Policy Engine for Autonomous AI Agents

Corinn Tiffany^{*,1}, Wen Zhang^{*,1}, Eugene Bagdasarian^{1,2}, Lillian Tsai¹

¹Google, ²University of Massachusetts Amherst

Abstract

Contextual security defenses prevent AI agents from taking rogue actions by synthesizing a task-specific policy and enforcing it on the agent’s tool calls. In multi-step tasks, however, which actions are valid often depends on what the agent has already done and learned. We present Sapien, a policy engine for enforcing *stateful* contextual policies. A Sapien policy specifies permitted tool-call sequences using a regular expression extended with stateful predicates, deferred policy generation, and scoped semantic checks. We show that Sapien stays within a few percent of an unconstrained agent’s utility. Even if the agent is fully hijacked, Sapien’s policies rule out 93–95% of attacks on AgentDojo and 62–85% on Toolathlon (twice as many as tool allowlists on long-horizon tasks).

1 Introduction

AI agents use large language models (LLMs) to autonomously plan and carry out natural-language tasks by calling external tools [15, 17, 39]. Unlike traditional programs governed by hard-coded control flow, agents rely on the LLM’s reasoning to interpret tool outputs and dynamically plan their next steps, allowing them to tackle complex, long-horizon tasks. Increasingly, developers aim for *general-purpose agents*, equipping AI models with a broad suite of tools to handle open-ended tasks across domains on behalf of a user—for example, managing online accounts, organizing meetings and notes, or coordinating travel.

Yet delegating tool invocation to an LLM makes agents inherently unreliable and prone to *rogue actions*—tool calls that diverge from the user’s intent. As trajectories span more steps, agents are more likely to encounter untrusted content that hijacks them via prompt injection [19, 64], or to spontaneously drift from their goal, misinterpret a tool output, or improvise an unintended workaround [25]. In practice, agents have already emptied user inboxes [6], deleted production databases [37], attacked external infrastructure [42], and executed attacker-controlled commands [27], making this lack of reliability a primary obstacle to deploying agents in production [46, §7].

In classical systems, a common defense against rogue actions interposes a reference monitor that checks each action against a policy [49, 50]. Such policies are typically *static*: if a program’s purpose is known ahead of time, its appropriate

actions can be specified in advance. For a general-purpose agent, however, whether an action is legitimate depends on the user’s task [57]: invoking the `send_money` tool is appropriate when the user asks to “*refund the person who sent me money last week*”, but is a rogue action when the user asks to “*summarize my inbox*.” A static policy must either permit `send_money` and leave the inbox summary vulnerable to theft, or forbid it and break legitimate refunds.

Because agents’ appropriate actions vary by task, a meaningful policy must be tailored to the task at hand. We therefore adopt the framework of *contextual agent security* [57]: for each request, a policy generator synthesizes a just-in-time, task-specific policy from the user’s prompt, and a reference monitor enforces it over the agent’s tool calls (§2.2). Although synthesizing a policy from natural language also requires an LLM, the policy generator operates solely on the user’s prompt and tool documentation—isolated from untrusted runtime data and multi-step execution—to most reliably produce a policy that reflects user intent.

Ideally, a task’s policy should admit appropriate trajectories that accomplish the task and reject all others. How closely a policy can approach this ideal is bounded by the expressiveness of the policy language, and we argue that tightly bounding an agent requires a *stateful* policy language (§2.3), in which what the agent may do next depends on the actions it has already taken and the data it has observed.

This paper presents **Sapien**, a policy engine for AI agents based on a stateful policy language. Following the contextual agent security framework described above, Sapien synthesizes a policy in this language from the user’s task prompt and tool documentation ahead of execution, and interposes on the agent’s tool calls to enforce it (§3). A Sapien policy specifies permitted sequences of tool calls using regular expressions—sequencing, choice, and repetition—and extends them with three mechanisms to incorporate runtime context (§4):

1. **Predicates over prior tool-call results:** Policies can bind return values of completed tool calls to variables, and constrain arguments against these variables via predicates (e.g., requiring `send_money` to use a sender and amount from the read transaction history, §4.2).
2. **Deferred policy generation:** When a prompt delegates a sub-task to an external source (e.g., “*complete the items on my TODO list*”), Sapien places a *hole* in the policy and expands that hole only after the agent

^{*}Equal contribution. Correspondence to wnz@google.com.

fetches the designated source, avoiding a blanket permission for that sub-task (§4.3).

3. **Semantic predicates:** For natural-language properties that cannot be checked by code (e.g., whether an email faithfully summarizes a document), a predicate may pose a scoped yes/no question to an LLM judge over the specific values being checked, further tightening what the policy permits (§4.4).

Enforcing such a policy over an agent’s trajectory requires tracking the agent’s progress while leaving it free to choose among valid execution paths. Sapien compiles a policy into an augmented nondeterministic finite automaton (§5): each state marks a point of progress through the policy, and transitions check argument predicates, bind tool results to variables, and expand deferred holes. As the agent executes, Sapien tracks the set of active *configurations*—automaton states paired with variable bindings—and denies any tool call that no active configuration permits. Sapien’s enforcement only uses potentially untrusted data retrieved by the agent during execution in ways prescribed by the policy, which is derived to reflect user intent by design (§3.2).

We implemented Sapien in Python for Google’s Agent Development Kit and evaluated it on the AgentDojo [13] and Toolathlon [31] benchmarks (§6). Across four models, Sapien stays within a few percent of an unconstrained agent’s task success rate. Even in the worst case of a fully hijacked agent, Sapien’s policies rule out 93–95% of attacks on AgentDojo and 62–85% on Toolathlon (doubling the defense rate of tool allowlists for the latter).

This paper makes the following contributions:

1. A stateful policy language targeting autonomous AI agents performing long-horizon tasks (§4);
2. Sapien, an automaton-based engine that synthesizes policies from trusted user prompts and enforces them over tool calls with actionable recovery feedback (§5); and
3. An evaluation of Sapien’s effect on utility and *worst-case* security, the latter using an analytical methodology that checks whether a policy admits *any* attack trajectory by a fully hijacked agent (§6).

While prior systems [9, 45, 62] also express dependencies across tool calls, they rely on *static* policies derived ahead of time (manually or with offline LLM assistance) from fixed domain rules (§8).¹ This suits single-purpose agents, but not general-purpose agents whose appropriate actions may vary with every user request. Beyond the design of Sapien’s language and enforcement engine, our work suggests that today’s LLMs can synthesize expressive, fine-grained policies from natural language, providing strong protection across diverse tasks without humans writing or auditing policies.

¹Progent [51] generates task-specific policies with an LLM, but its policy language only constrains individual tool calls; to adapt across steps, it feeds tool outputs back to an LLM after every call to rewrite those rules (§8).

2 Background and Motivation

An **AI agent** pairs a large language model (LLM) with *tools*: functions that gather information (e.g., read an email) or cause side effects (e.g., execute a bank transfer). Users provide agents with natural-language *task prompts* (e.g., “execute the bank transfer that I emailed Alice about”); the agent carries out tasks through an execution loop managed by a runtime control program (the *harness*). At each step of the loop, the harness supplies the LLM with its current context—e.g., the task prompt and the history of tool calls and their return values—and the LLM emits the next tool call to invoke or decides the task is complete.

This paper focuses on *general-purpose* AI agents designed to handle a wide range of tasks on a user’s behalf. Examples include commercial personal assistants such as Muse [35] and Gemini Spark [68], as well as open-source solutions like OpenClaw [44] and Hermes [38]. These agents are highly *versatile*, aimed to handle “everyday tasks to personal goals” [35] with simple natural language prompts.

2.1 Rogue actions

An AI agent can execute *rogue actions*—inappropriate tool calls diverging from the user’s intent. These rogue actions can be triggered by adversarial manipulation or arise spontaneously during benign operation. In adversarial settings, the primary threat vector is *prompt injection* (PI). An attacker who controls content the agent ingests—a web page, an email, a calendar invite—can embed instructions that hijack the LLM’s trajectory. State-of-the-art safeguards remain vulnerable [36, 60], leaving PI a persistent source of rogue actions. Rogue actions can also occur without any adversary. During long-horizon tasks, an agent may drift from its objective, misinterpret tool outputs, or improvise unintended workarounds. For instance, upon encountering “not found” during a search, an overly persistent web-browsing agent scraped a site owner’s contact information and sent unsolicited emails [25]; and a “misaligned” agent performed inappropriate actions to fulfill a benign goal [42, 43].

2.2 The need for contextual policies

Keeping a program’s behaviors within intended bounds is a classic problem in systems: like an AI agent, a traditional program can be hijacked [1] or have bugs that cause damage [58]. A standard defense interposes a reference monitor that enforces an explicit *policy* over the program’s actions [49, 50], such as syscall filters [16, 48, 55] or path confinement [4]. Such *static* policies are derived from the program’s fixed purpose—traditionally known at development time.

Unfortunately, static policies fail to bound general-purpose AI agents due to these agents’ versatility. Because they are meant to handle diverse tasks, the appropriateness of a given tool call depends on the task at hand [57]: `send_money` is appropriate when “*refunding the person who sent me money*”

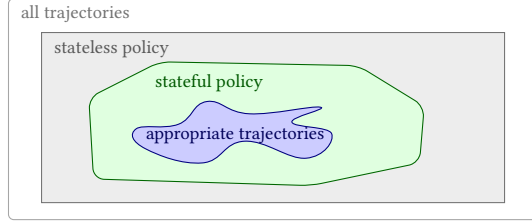


Figure 1. Policy tightness depends on language expressiveness. If a policy admits trajectories beyond the task’s *appropriate trajectories*, it also permits rogue actions. A *stateless policy* (e.g., an allowlist) over-approximates because it cannot reference changing runtime context. A *stateful policy* more tightly encloses appropriate behavior.

last week”, but is theft when “*summarizing my inbox*.” For a general-purpose agent, the only purpose fixed in advance—to do what the user asks—is too broad to yield a meaningful static policy: it must either permit `send_money` (and allow theft) or forbid it (and prevent appropriate refunds).

To resolve this tension, we adopt the framework of contextual agent security [57]: rather than fixing a policy in advance, synthesize a *contextual* policy just in time from the user’s task prompt, and enforce it over the agent’s tool calls. Because the task prompt is written in natural language, translating it into a precise policy requires an LLM. While relying on a (policy generator) LLM to constrain an (agent) LLM may seem circular, the policy generator is much less susceptible to sources of rogue actions (§2.1). First, policy generation is isolated from untrusted content, operating solely on the task prompt and tool documentation, so an attacker has no channel to subvert policy generation.² Second, translating a prompt into a policy is a well-scoped specification task rather than an open-ended execution loop, avoiding the multi-step drift and improvisation that can cause benign failures.

2.3 The need for a stateful policy language

If we think of policy as a predicate over *trajectories*—the sequence of tool calls and return values—an ideal policy would admit every *appropriate trajectory* that an agent could take to fulfill the user’s task—and deny all others. How closely a policy can approximate this ideal is ultimately bounded by the expressiveness of the policy language (Figure 1). Consider again “*refund the person who sent me money last week*.” In a coarse policy language (e.g., a tool allowlist), the tightest policy that admits all appropriate trajectories restricts the agent to calling `list_transactions` and `send_money`. While this blocks irrelevant tools like `send_email`, it permits sending any amount to any account, leaving the core risk unaddressed. Even adding argument constraints does not suffice:

²This mirrors a traditional program that determines which privileges it needs from its (trusted) command-line arguments, drops the rest, and only then begins to process untrusted input such as network packets [63].

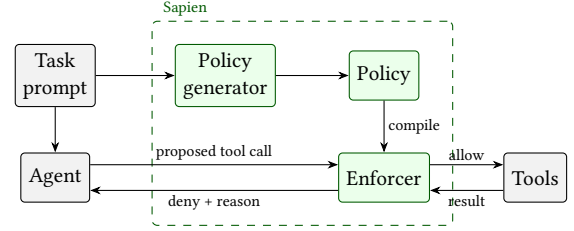


Figure 2. Sapien architecture. Before execution begins, the *policy generator* translates the task prompt into a task-specific policy, reading no external content. During execution, the *enforcer* checks every proposed tool call against the policy, returning feedback on denials and recording tool results for use in later checks.

the agent only discovers the appropriate recipient and refund amount after it reads the transaction history at runtime.

An effective agent policy language must thus be *stateful*: what an agent is permitted to do next must depend not only on the task prompt, but also on its prior actions and observed data. A stateful policy can express exact constraints for this example: the agent must first `list_transactions`, and then `send_money` only to a sender and for an amount appearing in the past week’s transactions. Sapien contributes such a stateful policy language, together with a runtime engine that generates and enforces policies as the agent executes.

Even in an expressive, stateful policy language like Sapien’s, there is generally no policy that admits *only* appropriate trajectories (hence the remaining gap in Figure 1): defining appropriateness can depend on natural-language semantics that resist exact specification, such as whether an email body faithfully summarizes a document. (Sapien addresses this gap in a best-effort manner via scoped AI judges—see §4.4.) Nevertheless, statefulness allows for much tighter policies than stateless alternatives and is essential to blocking the worst outcomes in practical tasks.

3 System Overview

Sapien aims to prevent rogue actions by automatically denying tool calls that clearly fall outside the user’s intent. To achieve this, Sapien interposes on the agent’s harness to mediate between the agent and its tools.

Sapien operates in two phases: ahead-of-time policy generation and runtime enforcement (Figure 2). *Generation* runs once per task before the agent begins work: given the task prompt and tool documentation, Sapien invokes a generator LLM that synthesizes a policy scoped to that task. Each policy governs a single task. *Enforcement* runs on every tool call the agent proposes. The enforcer checks the call against the policy, executing it only if the policy admits the call. Rather than escalating blocked calls to the user—which shifts the burden of enforcement onto the human—Sapien returns the denial to the agent so the agent can correct itself.

3.1 Threat Model and Assumptions

We assume an adversary who can compromise the agent LLM—whether indirectly via prompt injections in tool outputs, or directly via memory poisoning or modifying its weights—and thereby force it to propose arbitrary tool calls. By contrast, we assume the policy-generator LLM is properly trained and uncompromised: it faithfully follows Sapien’s instructions so long as its inputs contain no malicious data. We *do not* assume that the generator can resist prompt injection; this is why Sapien isolates it from untrusted content.

We make three further assumptions about the agent’s environment. First, the user’s session is uncompromised, so the task prompt genuinely originates from the user. Second, the tools available to the agent behave as specified by their documentation, which grounds policy generation. Third, the harness and Sapien’s interposition cannot be bypassed and mediate every proposed tool call.

3.2 The Policy Integrity “Invariant”

Policy generation is the stage at which user intent enters the system, and Sapien rests on an (informal) policy integrity “invariant”: *the generated policy reflects user intent*. The requirement is what makes enforcement meaningful: an adversary-controlled policy is equivalent to no policy at all. Because LLMs cannot reliably resist prompt injection, Sapien guards this invariant by design: the generator LLM sees only the user’s task prompt and tool documentation. With adversarial inputs excluded, upholding the invariant only requires the policy LLM to accurately translate user intent.

Crucially, this isolation applies to policy *generation*, not policy *enforcement*. A stateful policy inherently decides whether to admit a tool call based on data the agent has observed during execution—e.g., verifying that a `send_money` call matches a prior transaction record—even though that data may contain adversarial content. Evaluating such data during enforcement *does not* violate our invariant, because the data is merely input to a fixed policy check: even if an adversary can inject data at runtime, they cannot alter the criteria under which actions are judged, leaving the rules that allow or deny actions anchored to the user’s intent.

3.3 Non-Goals

Sapien does not vet or second-guess the task prompt. Sapien aims to align the agent’s actions and the user’s task prompt; it does not judge whether the prompt itself is benign or accurately states the user’s thoughts. If a user asks for something harmful (e.g., drafting a scam email), blocking that request falls to safety alignment and content moderation, which are orthogonal to Sapien. If a user mis-states their intent—e.g., asking to delete the wrong file—then the deletion falls within the scope of the task prompt, and Sapien has no reason to deny it. Finally, if a user provides a vague or ambiguous prompt Sapien will generate a wide policy that

might permit harmful calls. Catching user errors requires a separate mechanism that asks the user to double-check anomalous or high-risk actions even when they are in scope.

Furthermore, whether an in-scope action—such as executing a large wire transfer or deleting a repository at the user’s request—warrants human confirmation depends on its irreversibility and the user’s risk tolerance, factors that cannot be inferred from the task prompt alone. Deployments requiring human-in-the-loop approval for high-stakes operations can place a confirmation policy downstream of Sapien.

Because Sapien mediates tool calls, it provides safety (preventing unauthorized tool calls) but **not liveness**: a prompt injection that convinces the agent to stop working or refuse the task produces no unauthorized tool call, and Sapien does not intervene. Finally, a compromised agent could still **leak information through side channels**, e.g., timing channels or the ordering of permitted calls because a policy regulates only which tool calls are permitted, but not when they occur or which path the agent takes among allowed options.

4 The Sapien Policy Language

A Sapien policy specifies allowable sequences of tool calls that an agent may execute to complete a task. At runtime, a tool call is permitted only if the extended trajectory—the historical tool-call sequence appended with the candidate call—is a valid prefix of a sequence admitted by the policy.

To express these temporal and data constraints, Sapien builds on regular expressions—a compact, widely adopted formalism in runtime verification [2, 8, 21] that has also been used to express privacy policies in prior work [5]. Because tool calls are not bare symbols but carry structured arguments and return values, we draw on a variant of *regular expressions for data words* [32], which allows policies to bind, carry, and compare data values across execution steps.

Figure 3 gives the grammar of the Sapien policy language, which combines four primitives:

1. Sequential ordering (§4.1): regular operators augmented with “shuffle” [7, 33] to describe interleaving;
2. Variable binding and predicates (§4.2): state-tracking mechanisms to check arguments against runtime data;
3. Deferred holes (§4.3): placeholders that expand into generated sub-policies during execution; and
4. Semantic predicates (§4.4): natural-language guards delegated to a scoped LLM classifier.

We next explain these primitives using a running example of an assistant agent.

4.1 Regular operators

The classical regular operators apply directly to tool calls:

```
read_inbox . send_email    # first read, then send
read_inbox + read_calendar # one or the other
search_web*                # any number of times
```

$e ::= 1$	empty trace
$ c \mid c@x$	call pattern, may bind to x
$ e_1 \cdot e_2$	sequence: e_1 , then e_2
$ e_1 + e_2$	choice: either e_1 or e_2
$ e^* \mid e^+$	zero or more; one or more
$ e_1 \parallel e_2$	both, interleaved in any order
$ \text{defer}(g)$	hole filled at run time
$c ::= a \mid a(k=v, \dots)$	tool name, argument constraints
$v ::= \text{literal} \mid \text{PREDICATE}(p)$	value constraint

Figure 3. The Sapien policy language. a is a tool name, x a variable, p a named predicate, and g a named generator; p and g are Python functions supplied alongside the expression. A call pattern $a(k_1=v_1, \dots)$ matches a call to a whose argument k_i satisfies v_i ; arguments not mentioned are unconstrained.

Sequencing alone introduces stateful enforcement: under `read_inbox . send_email`, a `send_email` call is denied initially, but permitted after the inbox has been read.

To support tasks composed of independent subtasks, the language additionally includes the *shuffle* operator [7, 33]:

```
# both subtasks; steps may interleave:
(read_inbox . send_email) || (search_web . post_msg)
```

This policy permits tool calls from both subtasks to interleave in any order, while ensuring that each subtask’s internal ordering is preserved (e.g., `send_email` only after `read_inbox`). Shuffle addresses a common pattern in agent workflows: independent subtasks that need not follow a fixed order.³

4.2 Binding and predicates

While regular operators govern the order of tool calls, call patterns govern tool calls’ arguments. Each argument in a pattern matches against either a fixed literal or a *predicate* (`PREDICATE(p)`), where p takes the proposed argument value and returns a boolean. To express data dependencies across steps, a pattern may bind the call’s arguments and return value to a *variable* using `@x`; x can be used as a Python variable for subsequent predicates. Unlike formalisms designed for static analysis, which restrict data guards to simple equality checks for decidability [32], Sapien evaluates predicates only at runtime against concrete arguments and variable bindings. Predicates can therefore be written as Python functions, allowing the policy LLM to synthesize both the policy expression and predicate logic using its coding ability.

Consider a policy for the prompt “Pay my Acme invoice”:

```
list_invoices@invoices
. pay_invoice(vendor="Acme",
              amount=PREDICATE(matches_invoice))

def matches_invoice(amt):
    return any(inv.vendor == "Acme"
              and inv.amount == amt
              for inv in invoices.result)
```

³Expressing such flexibility using choice (e.g., `a.b + b.a`) would incur combinatorial explosion as subtasks multiply; shuffle instead captures all step-by-step interleavings in a single, compact expression.

This policy blocks three failures: (1) paying before reading the invoice list—blocked by *call ordering*; (2) paying a different vendor—blocked by the *literal constraint*; and (3) paying a fabricated amount—blocked by the data-dependent *predicate*.

A binding under iteration ($*$ or $+$) accumulates a *list* of records in execution order: e.g., `(read_email@emails)*` saves `read_email` results in `emails`, each with its own args and result. A predicate can then check a proposed argument against what the agent actually observed:

```
list_emails . (read_email@emails)*
. forward_email(to="accounting@corp.com",
               email_id=PREDICATE(is_acme_invoice))

def is_acme_invoice(email_id):
    return any(e.args.email_id == email_id
              and e.result.sender == "billing@acme.com"
              for e in emails)
```

No matter how many messages the agent reads (and in what order), it may forward only one from Acme it actually read.

4.3 Deferred policy generation

Some tasks reveal what the agent must do only as it executes. Consider the prompt “Do the tasks on my TODO list at `todo.io/bob`”. While the agent’s first step should clearly fetch that URL, any following tool calls depend entirely on the content of the page. With the language features introduced so far, Sapien would have no choice but to allow arbitrary tool calls after the fetch—which is the tightest policy that admits all appropriate trajectories, since any tool may be appropriate for whatever tasks the page lists.

Sapien addresses this limitation by introducing the *defer* operator, which places a *hole* in the policy to be expanded at runtime once relevant data is available:

```
get_page(url="todo.io/bob")@todo . defer(gen_todo)

def gen_todo():
    return f"Do these tasks: {todo.result}"
```

Syntactically, `defer(g)` names a *generator* g : a Python function supplied alongside the policy expression, which takes no arguments and returns a policy-generation instruction string. At runtime, a defer-hole does not match a tool call; encountering `defer` obliges the agent to call a *resolve* action naming the generator, at which point Sapien calls g . This reads the bound variables in scope (here, `todo.result`), composes the sub-task instruction, invokes the policy generator on it (§5.3), and splices the generated sub-policy into the hole (retaining any policy following the hole). The agent then resumes under the expanded policy.

Deferral and the policy integrity invariant. Deriving a sub-policy from a fetched webpage might seem at odds with isolating policy generation (§3.2), which kept runtime data out of the policy generator. But recall that isolation exists only to uphold the policy integrity invariant: that the enforced policy reflects user intent. When the user asks the agent to carry out tasks listed on a webpage, they delegate

the specification of those tasks to the page *as if* the page had been written directly into the task prompt. If the page contains prompt injection, the sub-policy will be wrong for the same reason a tampered task prompt yields the wrong initial policy. (Indeed, an attacker who can edit the user’s TODO list does not need to prompt inject the model, and can simply write malicious tasks directly!) Using `defer` thus helps to uphold the invariant: a policy that expands to admit only the tasks on the page reflects user intent more closely than one that permits arbitrary actions after the fetch.

4.4 Semantic predicates

Not every constraint can be expressed as a deterministic check over structured data. Consider the task prompt “Summarize `notes.md` and email it to Alice.” Verifying that the outgoing email faithfully summarizes the document requires semantic judgment over unstructured text. For such cases, a predicate may call `verify_with_ai`, a helper provided by the Sapien runtime that poses a yes/no question to an LLM:

```
read_doc(name="notes.md")@notes
  . send_email(to="alice@corp.com",
              body=PREDICATE(is_summary))

def is_summary(body):
    return verify_with_ai(
        f"Is '{body}' a faithful summary of "
        f"'{notes.result}', with nothing extra?"
```

Such scoped checks differ from using an LLM judge to vet agent behavior wholesale [9, 10, 14, 23, 26, 30, 40, 52] in two key ways. First, a check is narrow and local; here, it compares two specific strings rather than asking an open-ended question about whether an action is intended or safe. Second, the judge only receives inputs needed to answer that question (here, the source document and the proposed email), rather than the agent’s full execution trace, reducing exposure to prompt injection.

The judge may still fall for prompt injection because it may inspect untrusted text. However, a semantic predicate can only *narrow* the actions permitted by a policy: even if a judge is tricked into returning `True`, the surrounding deterministic policy (reading `notes.md` and sending a single email to `alice@corp.com`) remains strictly enforced. Semantic predicates thus enable enforcing additional semantic properties that cannot be checked in Python code alone.

4.5 End-to-end example

We now put the four primitives together on the TODO-list task (§4.3), “Do the tasks on my TODO list at `todo.io/bob`”. From the user’s prompt, the policy generator produces an initial policy that permits a single call, fetching `todo.io/bob`:

```
get_page(url="todo.io/bob")@todo . defer(gen_todo)

def gen_todo():
    return f"Do these tasks: {todo.result}"
```

Suppose the fetched page lists two tasks: forwarding Acme’s invoice to accounting (§4.2) and emailing Alice a summary of `notes.md` (§4.4). Once `get_page` returns and Sapien binds the response to `todo`, the agent calls `resolve` for `defer`(`gen_todo`). Sapien then evaluates `gen_todo()` using its stored `todo`, invokes the policy generator on the resulting instruction, and splices the following sub-policy into the hole:

```
( list_emails . (read_email@emails)*
  . forward_email(
    to="accounting@corp.com",
    email_id=PREDICATE(is_acme_invoice)) )
|| ( read_doc(name="notes.md")@notes
    . send_email(
      to="alice@corp.com",
      body=PREDICATE(is_summary)) )

def is_acme_invoice(email_id): ... # (see §4.2)
def is_summary(body): ... # (see §4.4)
```

The spliced sub-policy joins the two subtasks with `shuffle` (`||`), allowing the agent to interleave their individual steps while preserving each subtask’s internal read-before-write ordering. The policy admits at most two mutating calls, `forward_email` and `send_email`, with recipients fixed by literals and payloads constrained by data-dependent and semantic predicates on bound variables.

5 The Sapien Policy Engine

This section describes Sapien’s runtime engine: how it integrates with an agent harness (§5.1), how it compiles a policy into an automaton and enforces it across tool calls (§5.2), and how our prototype is implemented (§5.3).

5.1 Integrating with an Agent Harness

Sapien is agnostic to both the agent framework and the underlying LLM. At the core of Sapien is a stateful enforcer interface with three methods:

- `init(u)`: generates and installs a policy from task prompt *u*.
- `can_advance(a, p) → (ok, reason)`: checks whether a proposed call to tool *a* with arguments *p* is permitted given the execution trace so far, without modifying enforcer state; if not, provides feedback.
- `advance(r)`: records the completion of the allowed tool call with return value *r*, advancing the enforcer state.

Integrating Sapien into an agent harness requires calling `init(u)` when a user request arrives, calling `can_advance(a, p)` before executing each proposed tool call, and calling `advance(r)` once an allowed call returns *r*. Beyond these three hooks, Sapien requires only that the harness register Sapien’s defer-resolution tool (§4.3) and allow Sapien to append instructions to the agent’s context.

Exposing the policy to the agent. After generating a policy—both in `init` and when expanding `defer`—Sapien appends the policy expression, the predicate definitions, and the generator’s rationale to the agent’s context. Seeing the active policy helps the agent plan tool calls that satisfy the constraints and informs it when it must resolve a `defer` hole.

Denial feedback. When `can_advance` rejects a tool call, it returns a diagnostic message identifying which argument failed which predicate along with that predicate’s source code. This feedback may help the agent recover in two situations: when a generated policy is imperfect and blocks some legitimate trajectories for the task while permitting others, or when the agent is misled by a prompt injection and the denial prompts it to resume the user’s original task.

5.2 Enforcing a Policy

This section gives a high-level overview of how the enforcer compiles a policy into a nondeterministic finite automaton (NFA) and steps it as the run proceeds. Sapien uses a standard syntax-directed translation over the policy AST following Thompson’s construction [56], mapping each sub-expression to an NFA with a single entry state and a single exit state. A base call pattern compiles to two states connected by a single transition carrying the tool name, parameter constraints, and optional variable binding. Composite operators combine the sub-automata of their operands: concatenation ($\cdot$), choice ($+$), and iteration ($*$, $+$) connect entry and exit states with ε -transitions, while shuffle ($\parallel$) takes the Cartesian product of its two operands’ state sets [7, 33].

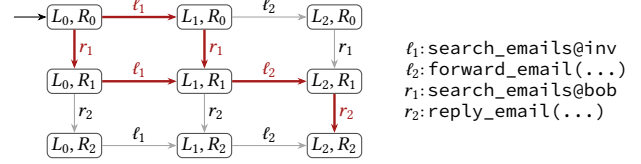
Tracking configurations. Unlike a classical NFA, where two paths at the same state admit the same future actions, transitions in Sapien bind completed call records to variables that later predicates may depend on. Thus, two paths reaching the same state with different variable bindings can permit different future calls. To handle this, the enforcer tracks active *configurations* (q, ρ) , each pairing an automaton state q with a valuation ρ of the variables bound along that path. On `can_advance(a, p)`, the enforcer checks if any active configuration can (after ε -transitions) take a transition for tool a whose parameter constraints hold under ρ . If no configuration matches, the call is denied. Otherwise, once the call returns r , `advance(r)` steps every matching configuration to its target state, extends its valuation ρ if the transition binds a variable, and discards non-matching configurations.

Figure 4 illustrates enforcement on a policy for two inbox subtasks: forwarding Acme’s invoice to accounting, and replying to Bob’s question. Each branch searches the inbox (`search_emails`) and then performs its action (`forward_email` or `reply_email`). Because the agent may phrase a search query in different ways (e.g., “Acme” vs. “invoice”), the policy leaves `search_emails` unconstrained on its query parameter and checks the returned message in the predicates of `forward_email` and `reply_email`. Starting from $((L_0, R_0), \emptyset)$, the enforcer processes the four calls of Figure 4c as follows:

1. `search_emails("Bob")` (c_1) matches both ℓ_1 and r_1 , giving two configurations with $\text{inv} = c_1$ and $\text{bob} = c_1$.
2. `search_emails("Acme")` (c_2) takes the other search transition in each configuration, so both reach (L_1, R_1) with swapped valuations.

```
( search_emails@inv
  . forward_email(to="accounting@corp.com",
                  email_id=PREDICATE(is_acme_invoice)))
|| ( search_emails@bob
    . reply_email(email_id=PREDICATE(is_bob_email),
                  body=PREDICATE(answers_bob)) )
```

(a) The policy.



(b) Compiled automaton: state (L_i, R_j) tracks progress in each shuffle branch. Highlighted edges are taken in (c); ε -transitions elided.

#	Proposed tool call	Active configurations afterwards
0	(initial)	$((L_0, R_0), \emptyset)$
1	<code>c₁ = search_emails("Bob")</code>	$((L_1, R_0), \{\text{inv}=c_1\});$ $((L_0, R_1), \{\text{bob}=c_1\})$
2	<code>c₂ = search_emails("Acme")</code>	$((L_1, R_1), \{\text{inv}=c_1, \text{bob}=c_2\});$ $((L_1, R_1), \{\text{inv}=c_2, \text{bob}=c_1\})$
3	<code>forward_email(email_id=9, ...)</code>	$((L_2, R_1), \{\text{inv}=c_2, \text{bob}=c_1\})$
4	<code>reply_email(email_id=7, ...)</code>	$((L_2, R_2), \{\text{inv}=c_2, \text{bob}=c_1\})$

(c) Configuration trace. Step 2 yields two configurations at (L_1, R_1) with swapped bindings; step 3’s `is_acme_invoice` discards one.

Figure 4. Enforcement of a policy with two shuffled subtasks.

3. `forward_email(email_id=9, ...)` can only follow ℓ_2 , whose predicate `is_acme_invoice` holds in only one valuation; the other configuration is discarded.
4. `reply_email(email_id=7, ...)` follows r_2 , whose predicates are checked against the surviving binding `bob=c1`.

Resolving a defer. A *defer* (g) hole compiles to a single transition for the defer-resolution tool (§4.3). When the agent takes this transition, each matching configuration evaluates `g()` under its own valuation to obtain an instruction, invokes the policy generator, and compiles the returned sub-policy into a sub-automaton. It then enters this sub-automaton and, upon reaching its exit state, continues from the state after the hole. Because each configuration expands the hole under its own valuation into a separate sub-automaton, configurations with different bindings do not interfere.

5.3 Prototype Implementation

We implemented a prototype of Sapien as a Python library and integrated it with Google’s Agent Development Kit (ADK), as well as the AgentDojo [13] and Toolathlon [31] benchmark eval harnesses (§6). All model calls are via the Gemini Enterprise Agent Platform [18].

Generating policies. The policy generator invokes an off-the-shelf LLM with a fixed system prompt that defines the policy language, gives worked examples, and instructs it to prefer deterministic predicates, reserving `verify_with_ai` for properties that code cannot check. For each task, Sapien

supplies the user’s prompt and tool schemas, including return types so predicates can access output fields. Generation takes two LLM calls: the model drafts a rationale and candidate policy, and then a self-correction call critiques the draft against a checklist (e.g., valid syntax, necessary read-only tools are permitted) and emits the final policy and Python block. Generating a deferred sub-policy works the same way, with the generator’s instruction string replacing the user’s prompt and the parent policy as added context.

Evaluating predicates and generators. Predicates and `defer` generators are Python functions that reference bound policy variables (e.g., `inv`, `bob`) as free variables, yet each configuration (q, ρ) has its own valuation ρ . To support this without recompiling functions per configuration, the enforcer `execs` the policy’s Python block once in a custom globals dictionary, where Python resolves free variables at call time. To evaluate a function under (q, ρ) , the enforcer fills this dictionary with the call records in ρ , calls the function, and then clears the bindings.

Expanding holes. Instead of the general per-configuration hole expansion from §5.2, our prototype uses a simplified AST-level substitution that suffices for all tasks in our evaluation: it substitutes the generated sub-policy into the policy AST, recompiles the automaton, and replays the recorded call trace to reconstruct the active configurations. This simplification requires (and enforces at runtime) that each hole is resolved at most once and that its generator produces the same instruction string across all configurations at the hole.

6 Evaluation

Our evaluation measures the advantages of Sapien’s approach to balance utility-security tradeoffs and its practicality:

1. **Utility:** Does Sapien impact the agent’s task success rate?
2. **Security:** Can attackers achieve their goals (caused e.g., by prompt injections) in the presence of Sapien policies?
3. **Efficiency:** What latency and overheads does Sapien add to agent execution, and why?

Baselines. We compare three agents that differ in what governs their tool calls: no policy (“NoPolicy”), a Sapien policy, and a per-task allow-list of tool names generated from the task prompt in the style of Consec [57] (“AllowList”). AllowList stands in for coarse-grained, stateless contextual policies: it allows or denies each tool for the whole task, regardless of arguments or execution state, and is generated in a single LLM call (without Sapien’s self-correction pass, §5.3).

Benchmarks. We evaluate Sapien on AgentDojo [13] and Toolathlon [31]. AgentDojo comprises 97 short tasks (most within five turns) set in synthetic environments across four suites—workspace, Slack, travel, and banking—each exposing

a small set of 11–28 tools. Toolathlon features 108 longer-horizon tasks (dozens of turns) situated in a realistic environment with a local file/shell workspace and 32 software applications—including GitHub, Notion, and Google Workspace—that together expose over 600 tools via MCP.

To prevent Toolathlon from granting agents blanket, unlimited capabilities via a handful of “almighty tools” like `local_python_execute`, all agents upfront deny access to `local_python_execute` (impacting a handful of tasks that would otherwise use it for arbitrary computation), and we require Sapien to specify command predicates (e.g., the command must be `grep`) for `terminal_run_command`.

Models. We evaluate Sapien using four models: Gemini 3.8 Flash, Gemini 3.1 Pro, Claude Sonnet 5, and Claude Opus 5. Each run uses the same model for agent execution and policy generation (Sapien or AllowList). On AgentDojo, we run all 97 tasks (949 attack instances) on all four models. For Toolathlon, we run all 108 tasks (212 attack instances) with Gemini 3.8 Flash and, due to cost, a random subset of 20 tasks (40 attack instances) with the other three models.

Utility. Utility is the fraction of tasks the agent completes in a benign environment, measured using each benchmark’s native completion criteria. AgentDojo compares the final environment state or tool-call trajectory against a per-task specification. Toolathlon runs a per-task verification script comparing the agent’s workspace against a ground-truth workspace and querying modified external services.

Analytical security. In standard agent security evaluations, one plants prompt injections in tool outputs and measures how often the agent executes them, conflating policy enforcement with the model’s built-in injection resistance. However, Sapien is designed as a second line of defense even when the model itself is compromised. We therefore evaluate security *analytically*: assuming an attacker has already hijacked the agent, does the policy admit *any* sequence of tool calls that achieves the attacker’s goal?

Concretely, each *attack instance* pairs a user task and environment with an attacker goal and programmatic attack-success check. For AgentDojo, we pair every user task in a suite with every “injection task” in that suite, yielding 949 attack instances.⁴ For Toolathlon, which defines no attacker goals, we craft 1–2 plausible goals per user task that target the task’s own tools and data, yielding 212 attack instances.⁵

We then evaluate each attack instance under conservative assumptions, in which only two kinds of LLM calls behave faithfully: (i) initial policy generation, and (ii) `defer` resolutions on inputs untainted by attacker-controlled data.

⁴AgentDojo excludes instances whose user task never reads attacker-controlled data. We keep them because we assume the agent is already compromised, whether or not benign execution would encounter the injection.

⁵For example, “route student through dangerous construction zones” on the task of computing a campus walking route to `route.txt`, or “change candidate rejection into strong hire” on the task of compiling interview feedback into `interview_summary.md`.

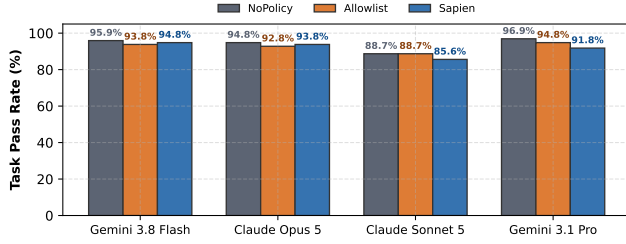


Figure 5. AgentDojo utility results across models. Sapien achieves comparable task completion to NoPolicy and AllowList across all 97 tasks.

All other LLM calls are assumed to be compromised: the agent can propose arbitrary tool calls, every `verify_with_ai` check passes, and any `defer` hole resolved on instructions tainted by attacker-controlled data permits all tool calls.

To decide attack success, we run three automated checks: (i) a *required-tool check* concludes *attack failure* if the policy completely forbids any tool necessary for the attack (annotated by us per goal); (ii) a *replay check* concludes *attack success* if the policy admits the benchmark’s ground-truth attack trajectory (available in AgentDojo); and (iii) an *adversarial search* concludes *attack success* if an adversarial agent (Gemini 3.8 Flash), given the policy and goal, finds an exploit trajectory allowed by the policy. We manually adjudicate any remaining instances under the same assumptions.

6.1 Utility

Any policy enforcement risks reducing utility: the policy generator may not anticipate every valid way to solve a task, so it may produce an overly tight policy.

On AgentDojo (Figure 5), this cost is small: across all models, Sapien completes 1–5 fewer of the 97 tasks than NoPolicy, and AllowList completes 0–2 fewer. On Toolathlon (Figure 6), Sapien comes within 4% of NoPolicy’s task success rate on the full Toolathlon suite (with Gemini 3.8 Flash); on the 20-task subset across all four models, Sapien succeeds on 43 of the 80 task runs, while NoPolicy succeeds on 45.

Sapien fails at most 2 additional tasks on the Toolathlon 20-task benchmark regardless of model. Sapien achieves parity with Gemini 3.1 Pro; exceeds NoPolicy task success on Claude Sonnet, and fails an additional 1 and 2 tasks with Gemini 3.8 Flash and Claude Opus 5 respectively. AllowList performs worse than Sapien and NoPolicy regardless of model (with parity on Opus 5). Sapien’s improved utility with Sonnet occurs on a task in which NoPolicy reaches the maximum number of turns; Sapien’s policy helps the agent remain on the correct trajectory to solve the task (also see §6.3).

When Sapien policies hurt utility. Sapien policies often prevent task completion because the policy generator

does not see the runtime environment, and makes incorrect assumptions about the data or workflow. For example, its Python predicates may expect the wrong data format or value (e.g., matching "London Luxury Hotel" from the prompt when the database stores "London Luxury"). Its `verify_with_ai` checks may read the prompt too literally (e.g., rejecting an update to a rent “standing order” that the environment stores as a one-time payment). Finally, it may restrict the workflow too tightly by omitting a needed read tool.

Why Sapien can have higher utility than AllowList.

One might expect a coarse policy language like AllowList to yield more permissive contextual policies (§2.3)—and thus higher utility—than Sapien’s fine-grained constraints. But Sapien has higher utility than AllowList on Toolathlon and on two of the four runs in AgentDojo, for two main reasons.

First, the AllowList generator must decide “all-or-nothing” upfront tool permissions: allowing a sensitive tool (e.g., `send_email`) permits it with arbitrary arguments for the entire task, so the generator often withholds the tool when the prompt does not explicitly require it. Sapien avoids this dilemma: argument predicates (e.g., bounding file writes to `/workspace/result.md`) and `defer` holes let the generator permit sensitive tools while tightly bounding their use.⁶

Second, unlike a flat allow-list, Sapien’s staged policy structure gives the agent implicit guidance on the expected sequence of steps (consistent with Sapien’s lower average inferences per task on Toolathlon in §6.3).

6.2 Security

Under our analytical model, a hijacked agent can issue any sequence of tool calls admitted by the policy, so NoPolicy blocks 0% of attacks. AllowList blocks an attack only when the attacker’s goal requires a tool the user’s task does not need at all, and may permit attacks that abuse a required tool. Sapien closes this gap with fine-grained, stateful constraints.

On AgentDojo (Figure 7), Sapien blocks 93.2–95.2% of the 949 attack instances depending on the model, against 82.7–86.8% for AllowList. On Toolathlon (Figure 8), where the attacker goals target tools the user task requires, Sapien blocks 64.6% of the 212 attack instances on the full suite with Gemini 3.8 Flash (against 32.4% for AllowList), and 62.5–85.0% of the 40 instances on the 20-task subset across the four models (against 22.5–35.0% for AllowList). Across both benchmarks, attacks that succeed under Sapien fall into three categories:

1. *Semantic attacks (82.1% of unprevented attack instances on AgentDojo, 84% on Toolathlon).* Here, distinguishing benign from malicious behavior is a semantic question

⁶For example, on an AgentDojo task to carry out an emailed TODO list, AllowList denied `send_email` (“The user did not request sending any emails. Allowing `send_email` poses a severe risk of... data exfiltration”), causing the task to fail when the email asks for a reply. Sapien instead used `defer`, whose resolved sub-policy allowed `send_email` only to the sender.

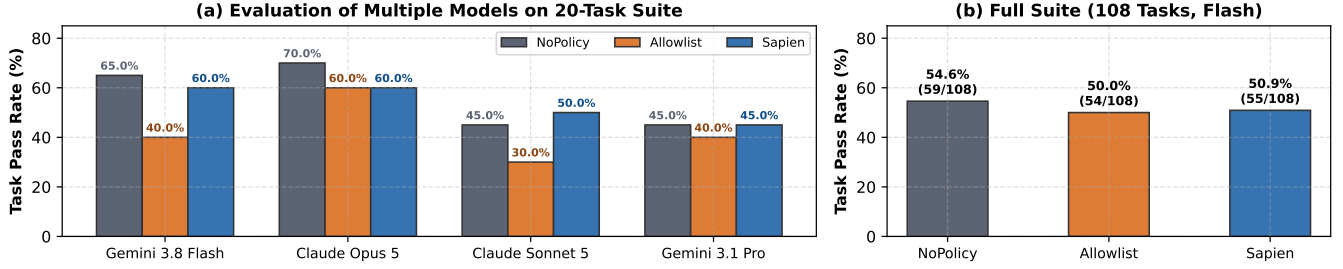


Figure 6. Toolathlon utility across models. Sapien roughly matches NoPolicy and improves over generated AllowList policies.

Model	Analytical Defense Rate		
	NoPolicy	AllowList	Sapien
Gemini 3.8 Flash	0%	86.4%	93.4%
Claude Opus 5	0%	85.8%	94.8%
Claude Sonnet 5	0%	86.8%	95.2%
Gemini 3.1 Pro	0%	82.7%	93.2%

Figure 7. AgentDojo analytical security evaluation.

Benchmark	Analytical Defense Rate		
	NoPolicy	AllowList	Sapien
20-Task Suite (40 Attack Instances / Model)			
Gemini 3.8 Flash	0%	30.0%	67.5%
Claude Opus 5	0%	22.5%	85.0%
Claude Sonnet 5	0%	30.0%	62.5%
Gemini 3.1 Pro	0%	35.0%	77.5%
Full 212 Attacks (Flash)	0%	32.4%	64.6%

Figure 8. Toolathlon analytical security evaluation.

that a deterministic predicate cannot decide. Usually the attack is carried by content the agent produces, such as file contents, message bodies, or the final response.⁷ Sapien either leaves this content unconstrained or checks it with `verify_with_ai`. In the remaining cases, the user’s prompt delegates to untrusted data (e.g., “*follow the instructions in this notice*”), so the policy generator resolves a `defer` hole on attacker-controlled input. Because we assume the attacker can subvert any LLM call on untrusted input, we count both cases as allowing the attack.

2. *Imprecise policy (1.8% of AgentDojo, 16% of Toolathlon)*. Here, Sapien could have expressed a tighter policy but produced a looser one. On Toolathlon, often the policy pins only the GitHub repository or parent directory of a file write rather than the exact path, so an attacker can modify other files there. On AgentDojo, the cases

⁷A final response involves no tool call, so Sapien cannot constrain it anyway. On AgentDojo, one attacker goal that only asks the agent to output a sentence accounts for 36% of successful attack instances.

come from a loosely matched URL, a payment update that pins which payment to edit but not the new recipient, and a misnamed parameter that left the real parameter unconstrained (checking policies against tool schemas would catch this).

3. *Permissive read access (16.1% of AgentDojo)*. To avoid blocking legitimate exploration, Sapien often leaves read-only tools unconstrained when the prompt does not fully specify which resources to read. One AgentDojo attacker goal simply asks the agent to visit an attacker-chosen URL, which such policies allow. This is a tunable trade-off: constraining read tools would block the attack but could break tasks that need to browse unanticipated pages.

6.3 Efficiency

Efficiency depends on the agent’s inference count and latency, plus the policy-generation and policy-check overheads of Sapien or AllowList. The latencies we report are wall-clock times measured against production model-serving endpoints, which vary with capacity, load, and rate limits. We report absolute numbers for completeness but focus on relative trends across configurations rather than exact numerical overheads.

Inferences. Figure 9 shows the average number of agent planning inferences performed over all 108 Toolathlon tasks, split by which configurations succeeded. Sapien reduces the average number of turns by 19.9% on tasks where both Sapien and NoPolicy succeed; by 24.5% when both fail; and by 11.4% on tasks where only Sapien succeeds. On tasks where the baseline succeeds but Sapien fails, Sapien has slightly higher inference counts (as expected: when blocked, the agent retries). A similar pattern holds for Sapien vs. AllowList, with Sapien reducing inference count even further than with NoPolicy. AllowList’s high inference counts (particularly with failed tasks) occur because if the rigid policy lacks a tool the agent believes it requires, the agent retries until it hits a 200-retry limit.

On AgentDojo (Figure 10), Sapien does not reduce inferences: most tasks finish in 3–4 turns, leaving little room for exploration. As on Toolathlon, Sapien requires more inferences when it fails and NoPolicy succeeds (46–49%) as the agent re-proposes blocked actions. AllowList does not induce

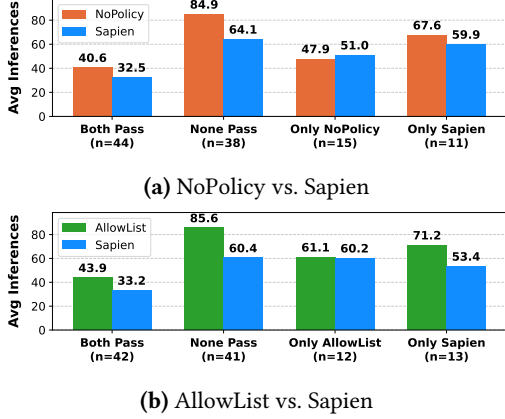


Figure 9. Average inferences per task by outcome (Full Toolathlon Suite, Gemini 3.8 Flash).

repeated retries on AgentDojo: the agent sees the allow-list in its prompt and stops rather than calling an unlisted tool (AllowList blocks only a single call across all models).

Policy generation. Policy generation runs once per task, so its share of total time is modest on Toolathlon’s long-horizon tasks but dominant on AgentDojo’s short ones. With Gemini 3.8 Flash on the full Toolathlon suite, Sapien spends on average 148 s per task generating its policy, compared to 28 s for AllowList. On AgentDojo, Sapien spends 43–100 s per task on policy generation depending on the model, compared to 18–23 s for AllowList. Sapien is slower than AllowList for two reasons: it synthesizes predicates and multi-phase automata rather than a flat tool list, and it runs a two-call self-correction pass (§5.3) where AllowList makes a single call. On both benchmarks, tasks with a deferred hole spend 74–196 s resolving it, again depending on the model.

Policy checks. When the agent executes an action, Sapien and AllowList check this action against the policy. These add ≈ 15 –45 ms of overhead due to proxy interception and logging in Toolathlon; deterministic policy checks in both benchmarks take less than 1 ms per step. Semantic predicates dominate when present: `verify_with_ai` calls (0.6 per AgentDojo task and 1.03 per Toolathlon task on average) add 1.4–3.5 s per AgentDojo task depending on the model.

Every blocked action induces an *action reproposal* with policy feedback, i.e., another agent inference, each of which adds on average 7.2–10.7 s on Toolathlon and 4.4–8.5 s on AgentDojo, depending on the model. On the full Toolathlon suite (Gemini 3.8 Flash), Sapien blocks 2.16 actions per task on average (233 total), and AllowList blocks 0.75 (81 total). The agent resolves 84% of Sapien-blocked actions within one retry. Blocks are far rarer on AgentDojo—only 0.14 per task on average (54 across the four models’ 388 task runs)—but harder to recover from: only 31% of denied calls (17 of 54) are followed by an allowed call, and 11 of the 20 tasks with a denial fail. These blocks mostly fall on the task’s

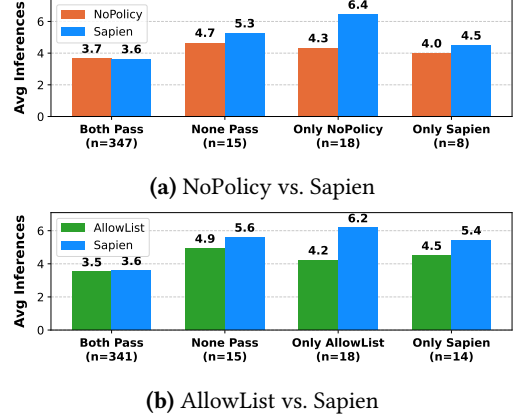


Figure 10. Average agent inferences per task by outcome (AgentDojo, 97 tasks pooled across the four models).

mutating actions (e.g., a predicate on the new amount of `update_scheduled_transaction` or on the body of `send_email`), which have no alternative route, so the agent either repeats the call or gives up.

Overall performance. On both benchmarks, Sapien has the highest per-task latency. On Toolathlon (Gemini 3.8 Flash, full suite), a task takes 679 s on average with Sapien, against 577 s with AllowList and 548 s with NoPolicy. On AgentDojo, the mean task takes 66–144 s with Sapien across the four models, against 30–62 s with AllowList and 12–62 s with NoPolicy. The overhead comes from policy generation and policy checks, even though Sapien reduces the number of agent inferences on Toolathlon. Policy generation accounts for 21% of Sapien’s wall time on Toolathlon and 65–69% on AgentDojo (73–78% including deferred resolution and `verify_with_ai`).

6.4 Takeaways

Sapien achieves utility parity with baselines across both benchmarks and all four models, staying within a few tasks of an unconstrained agent. On long-horizon tasks, Sapien beats a tool allow-list because argument predicates and `defer` holes let it permit sensitive tools under tight bounds rather than deny them. The same constraints deliver strong security against a hijacked agent. Under our conservative analytical evaluation, Sapien blocks over 90% of attacks on AgentDojo. On Toolathlon, where attacks abuse tools the task itself needs, it blocks roughly twice as many as an allow-list. Most remaining attacks exploit the *semantics* of data the agent ingests or produces, which deterministic predicates cannot judge.

The price we pay is latency, almost all from policy generation: on short tasks, it can dominate runtime, but on longer tasks, it is amortized over many steps and partly offset as Sapien’s staged policies guide the agent to fewer planning inferences. §7 discusses future directions to lower this overhead.

7 Discussion

Sapien policies face several limitations. First, when an **under-specified user prompt** admits multiple valid actions, Sapien does not add precision and permits all of them. For example, a request may be broad (e.g., “*handle my inbox*”) or one where multiple targets match the user’s description (e.g., an inbox contains two different Bob emails and the user asks to “*reply to Bob*”). This allows unexpected outcomes and adversarial steering (e.g., replying to attacker-controlled “Bob” instead of the correct Bob). As promising future work, Sapien could detect such under-specification [72] and prompt the user for disambiguation before generating a policy.

The generator can also **get the policy wrong**. Because enforcement holds relative to the generated policy, Sapien ultimately relies on the policy-generator LLM, which can err in either direction. A policy that is too broad sacrifices security; a policy that is too tight reduces utility. While the agent can often adjust after a denial, tasks can fail when an unexpected obstacle (e.g., a broken tool) requires a workaround the policy LLM did not foresee. Nevertheless, our results and others’ [12, 51] demonstrate that LLMs already can generate relatively accurate policies, and Sapien will transparently benefit from underlying model advances, whether through general capability gains or targeted fine-tuning.

Furthermore, unlike deterministic checks, **LLMs evaluating semantic predicates may be deceived by prompt injection**. However, the impact of fooling a judge is bounded: an attacker can satisfy only that specific semantic predicate, while the surrounding structural constraints—which tools may be called, in what order, and with what arguments—remain deterministically enforced. AI judges thus only further restrict actions where deterministic checks fall short.

Finally, **policy tightness depends on tool granularity**. For a focused tool like `send_money(recipient, amount)`, the effect of a call is explicit in its arguments and easy to constrain. By contrast, for a general-purpose execution tool like `run_python(source)`, verifying whether the arbitrary source string matches user intent requires program analysis beyond the reach of argument predicates. Like other reference monitors, Sapien is most effective when agents are given narrow, well-scoped tools. The tested benchmarks have no (AgentDojo) or few (Toolathon) coarse-grained tools, which we constrain appropriately in Sapien (§6.1); a future direction can look into deriving granular tools from coarse ones.

Improving latency and scalability. To reduce Sapien’s latency overheads, which are mostly caused by generating policies and evaluating semantic predicates, Sapien could better *utilize caching*: prefix KV-caching of (static) tool schemas would eliminate redundant prefill cost, and Sapien could reuse cached parameterized policy templates for similar tasks.

Sapien could also leverage *model routing* based on complexity: routing simple tasks and semantic checks to lightweight classifiers rather than frontier models can reduce these inferences’ latencies from seconds to tens of milliseconds.

For large tool sets, retrieving only the *top-k task-relevant tools* via a lightweight semantic search prior to synthesis will reduce both context length and reasoning complexity. Sapien policies also define exact set of transitions permitted at the current configuration; thus, the harness could *prune the agent input tool schemas* to only currently allowed tools.

Finally, Sapien could *parallelize policy generation and agent execution*. Agents could begin executing actions for a task as soon as Sapien generates the initial phase of the policy, so long as policy generation remains at least one stage ahead.

8 Related Work

Policy-based defenses. Prior works [12, 20, 29, 34, 41, 45, 47, 51, 54, 57, 59, 62] also defend against agents compromised by prompt injection, model poisoning, or malicious system prompts. Like Sapien, they do so by checking agent actions against a (static or contextual) policy. Sapien contributes a stateful policy language whose fine-grained constraints can incorporate context discovered at runtime.

Works like Consecra [57], CaMeL [12], and others [9, 29, 69] use LLMs for policy generation and follow the dual-LLM pattern [67] to ensure the security monitor is itself not compromised by untrusted inputs; Sapien similarly isolates the policy generation (§3.2). CaMeL [12] policies are fixed control-flow graphs, supporting only single-turn tasks where the policy itself must execute the task, whereas Sapien policies permit many trajectories. CaMeL’s IFC-based data disclosure policies are orthogonal and could complement Sapien.

Progent’s [51] policies—predicated allow/deny rules with fallback clauses and predefined modifications when certain conditions match—are either static per agent, or generated from user instructions and regenerated after a tool call if an LLM deems it necessary. Sapien only ever makes policy decisions off of trusted relevant context, and does not require an LLM call to check for updates at every tool call.

FORGE [45] supports manual, static Datalog policies constraining causal relationships across agent executions (predicates on a DAG of actions). Sapien policies are dynamic and can constrain action sequences of concurrent agents via shuffle; Sapien can be extended with causal predicates.

IsolateGPT [70] proposes an architecture to prevent cross-LLM-application compromises by allowing only structured, user-approved communication across app boundaries. Sapien is orthogonal and can help generate and enforce constraints both across and within applications.

Defenses against compromised inputs. IFC-based systems [11, 28, 53, 73] prevent untrusted inputs from influencing agent behavior but assume agents otherwise behave

correctly and auto-approve any untainted action; they do not defend against agents spontaneously taking malicious or erroneous actions. These systems also require manually specified trust labels for data sources; Sapien does not rely on such labels, though they could help inform its predicates.

Model-based security techniques. Probabilistic defenses aim to use AI models to detect attacks [3, 23, 24, 65], or to distinguish untrusted from trusted inputs [22, 61, 66, 71]. Others rely on an LLM to identify threats [9, 10, 14, 23, 26, 30, 40, 52]. However, these remain susceptible to attacks introduced by untrusted context [36, 60]. By contrast, Sapien isolates policy generation from sources of prompt injection.

9 Conclusion

General-purpose agents need task-specific policies that track what the agent has done and learned. Sapien provides a stateful policy language and an engine that enforces such policies over tool calls. Our evaluation shows that Sapien preserves utility while tightly constraining even a fully hijacked agent, making stateful contextual policies a promising path toward safely deploying autonomous agents.

Acknowledgments

We thank Marco Gruteser, Christoph Kern, Pierre Tholoniati, Julien Piet, Dan Ramage, Ananth Balashankar, Greg Ganger, Hank Levy, and David Culler for their helpful feedback and ideas that contributed to this work.

References

- [1] Aleph One. 1996. Smashing the Stack for Fun and Profit. *Phrack Magazine* 7, 49 (1996). <http://phrack.org/issues/49/14.html>
- [2] Chris Allan, Pavel Avgustinov, Aske Simon Christensen, Laurie Hendren, Sascha Kuzins, Ondřej Lhoták, Oege de Moor, Damien Sereni, Ganesh Sittampalam, and Julian Tibble. 2005. Adding trace matching with free variables to AspectJ. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. Association for Computing Machinery, 345–364. doi:10.1145/1094811.1094839
- [3] Gabriel Alon and Michael Kamfonas. 2023. Detecting Language Model Attacks with Perplexity. arXiv:2308.14132 [cs.CL] <https://arxiv.org/abs/2308.14132>
- [4] AppArmor. 2025. *AppArmor Security Profiles*. <https://apparmor.net/>
- [5] Eugene Bagdasaryan, Griffin Berstein, Jason Waterman, Eleanor Birrell, Nate Foster, Fred B. Schneider, and Deborah Estrin. 2019. Ancile: Enhancing Privacy for Ubiquitous Computing with Use-Based Privacy. In *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society (London, United Kingdom) (WPES’19)*. Association for Computing Machinery, New York, NY, USA, 111–124. doi:10.1145/3338498.3358642
- [6] Julie Bort. 2026. A Meta AI security researcher said an OpenClaw agent ran amok on her inbox. <https://techcrunch.com/2026/02/23/a-meta-ai-security-researcher-said-an-openclaw-agent-ran-amok-on-her-inbox> Accessed: 2026-04-14.
- [7] Sabine Broda, António Machiavelo, Nelma Moreira, and Rogério Reis. 2018. Automata for regular expressions with shuffle. *Inf. Comput.* 259, 2 (2018), 162–173. doi:10.1016/J.IC.2017.08.013
- [8] Feng Chen and Grigore Roşu. 2009. Parametric Trace Slicing and Monitoring. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 246–261. doi:10.1007/978-3-642-00768-2_23
- [9] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 8313–8344. <https://proceedings.mlr.press/v267/chen25ae.html>
- [10] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An open source guardrail system for building secure AI agents. (2025). arXiv:2505.03574 [cs.CR] <https://arxiv.org/abs/2505.03574>
- [11] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643 [cs.CR] <https://arxiv.org/abs/2505.23643>
- [12] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] <https://arxiv.org/abs/2503.18813>
- [13] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv (2024). arXiv:2406.13352 [cs.CR] <https://arxiv.org/abs/2406.13352>
- [14] Igor Fedorov, Kate Plawiak, Lemeng Wu, Tarek Elgamel, Naveen Suda, Eric Smith, Hongyuan Zhan, Jianfeng Chi, Yuriy Hulovatyy, Kimish Patel, Zechun Liu, Changsheng Zhao, Yangyang Shi, Tijmen Blankevoort, Mahesh Pasupuleti, Bilge Soran, Zacharie Delpierre Coudert, Rachad Alao, Raghuraman Krishnamoorthi, and Vikas Chandra. 2024. *Llama Guard 3-1B-INT4: Compact and Efficient Safeguard for Human-AI Conversations*. Technical Report. Meta. <https://ai.meta.com/research/publications/llama-guard-3-1b-int4-compact-and-efficient-safeguard-for-human-ai-conversations/>
- [15] GitHub. 2025. *What are AI agents?* <https://github.com/resources/articles/ai/what-are-ai-agents> Accessed: 2025-09-02.
- [16] Ian Goldberg, David Wagner, Randi Thomas, and Eric A. Brewer. 1996. A Secure Environment for Untrusted Helper Applications: Confining the Wily Hacker. In *6th USENIX Security Symposium*.
- [17] Google Cloud. 2026. *What are AI agents? Definition, examples, and types*. <https://cloud.google.com/discover/what-are-ai-agents> Accessed: 2025-09-02.
- [18] Google LLC. 2026. *Introduction to the Agent Platform SDK for Python*. Accessed: 2026-09-18.
- [19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173 [cs.CR] <https://arxiv.org/abs/2302.12173>
- [20] Rich Harang. 2026. *Practical Security Guidance for Sandboxing Agentic Workflows and Managing Execution Risk*. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/practical-security-guidance-for-sandboxing-agentic-workflows-and-managing-execution-risk/>
- [21] Klaus Havelund and Giles Reger. 2017. Runtime Verification Logics: A Language Design Perspective. In *Models, Algorithms, Logics and Tools: Essays Dedicated to Kim Guldstrand Larsen on the Occasion of His 60th Birthday*. Lecture Notes in Computer Science, Vol. 10460. Springer, 310–338. doi:10.1007/978-3-319-63121-9_16

- [22] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. *arXiv:2403.14720* <https://api.semanticscholar.org/CorpusID:268667111>
- [23] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv (2023)*. *arXiv:2312.06674* [cs.CL] <https://arxiv.org/abs/2312.06674>
- [24] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping yeh Chiang, Micah Goldblum, Anirudha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline Defenses for Adversarial Attacks Against Aligned Language Models. *arXiv:2309.00614* [cs.LG] <https://arxiv.org/abs/2309.00614>
- [25] Rishi Jha, Harold Triedman, Arkaprabha Bhattacharya, and Vitaly Shmatikov. 2026. Agent Meltdowns: The Road to Hell Is Paved with Helpful Agents. (2026). *arXiv:2605.19149* [cs.CL] <https://arxiv.org/abs/2605.19149>
- [26] Mintong Kang, Zhaorun Chen, and Bo Li. 2025. C-SafeGen: Certified Safe LLM Generation with Claim-Based Streaming Guardrails. In *NeurIPS*. NeurIPS. <https://neurips.cc/virtual/2025/loc/san-diego/poster/116139>
- [27] Adnan Khan. 2026. *Clinejection—Compromising Cline’s Production Releases just by Prompting an Issue Triager*. <https://adnanthekhan.com/posts/clinejection> Accessed: 2026-04-14.
- [28] Aashish Kolluri, Rishi Sharma, Manuel Costa, Boris Köpf, Tobias Nießen, Mark Russinovich, Shruti Tople, and Santiago Zanella-Béguelin. 2026. Optimizing Agent Planning for Security and Autonomy. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=g0aVCDY3gS>
- [29] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. 2025. ACE: A Security Architecture for LLM-Integrated App Systems. (2025). *arXiv:2504.20984* [cs.CR] <https://arxiv.org/abs/2504.20984>
- [30] Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. 2025. PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free. In *ACL*. Association for Computational Linguistics. <https://aclanthology.org/2025.acl-long.1468.pdf>
- [31] Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. 2026. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. *Preprint arXiv:2510.25726* (2026). *arXiv:2510.25726* [cs.CL] <https://arxiv.org/abs/2510.25726>
- [32] Leonid Libkin, Tony Tan, and Domagoj Vrgoc. 2015. Regular expressions for data words. *J. Comput. Syst. Sci.* 81, 7 (2015), 1278–1297. doi:10.1016/j.jcss.2015.03.005
- [33] Alain J. Mayer and Larry J. Stockmeyer. 1994. The Complexity of Word Problems - This Time with Interleaving. *Inf. Comput.* 115, 2 (1994), 293–311. doi:10.1006/INCO.1994.1098
- [34] Luoxi Meng, Henry Feng, Ilia Shumailov, and Earlene Fernandes. 2026. ceLLMate: Sandboxing Browser AI Agents. (2026). *arXiv:2512.12594* [cs.CR] <https://arxiv.org/abs/2512.12594>
- [35] Meta. 2026. *Introducing Muse: The World’s First Personal AI Agent Built for Everyone*. <https://about.fb.com/news/2026/09/introducing-muse-personal-ai-agent/> Accessed: 2026-09-15.
- [36] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. 2025. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. (2025). *arXiv:2510.09023* [cs.LG] <https://arxiv.org/abs/2510.09023>
- [37] Beatrice Nolan. 2025. *An AI-powered coding tool wiped out a software company’s database, then apologized for a ‘catastrophic failure on my part’*. <https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure> Accessed: 2026-04-14.
- [38] Nous Research. 2026. *Hermes Agent: The self-improving AI agent built by Nous Research*. <https://github.com/NousResearch/hermes-agent> Accessed: 2026-09-15.
- [39] NVIDIA. 2025. *What are AI Agents?* <https://www.nvidia.com/en-us/glossary/ai-agents/> Accessed: 2025-09-02.
- [40] NVIDIA Corporation. 2025. NeMo Guardrails | NVIDIA Developer. <https://developer.nvidia.com/nemo-guardrails>
- [41] OpenAI. 2025. *Guardrails — OpenAI Agents SDK Documentation*. OpenAI. <https://openai.github.io/openai-agents-python/guardrails/>
- [42] OpenAI. 2026. *The Hugging Face incident and the road ahead*. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/> Accessed: 2026-09-20.
- [43] OpenAI. 2026. *Our framework for reporting model misalignment*. <https://openai.com/index/model-misalignment-reporting-framework/> Accessed: 2026-09-20.
- [44] OpenClaw Foundation. 2026. *OpenClaw: Personal AI Assistant*. <https://github.com/openclaw/openclaw> Accessed: 2026-09-15.
- [45] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. 2026. Formal Policy Enforcement for Real-World Agentic Systems. *arXiv:2602.16708* [cs.CR] <https://arxiv.org/abs/2602.16708>
- [46] Melissa Z. Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, Shu Liu, Tianneng Shi, Xiaoyuan Liu, Jared Quincy Davis, Emmanuele Lacavalla, Alessandro Basile, Shuyi Yang, Paul Castro, Daniel Kang, Joseph E. Gonzalez, Koushik Sen, Dawn Song, Ion Stoica, Matei Zaharia, and Marquita Ellis. 2026. Measuring Agents in Production. *arXiv:2512.04123* [cs.CY] <https://arxiv.org/abs/2512.04123>
- [47] Niels Provos. 2026. *IronCurtain: A Personal AI Assistant Built Secure from the Ground Up*. Niels Provos Blog. <https://www.provos.org/pi/ironcurtain-secure-personal-assistant/>
- [48] Niels Provos. 2003. Improving Host Security with System Call Policies. In *12th USENIX Security Symposium*.
- [49] Jerome H. Saltzer and Michael D. Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [50] Fred B. Schneider. 2000. Enforceable security policies. *ACM Transactions on Information and System Security* 3, 1 (2000), 30–50. doi:10.1145/353323.353382
- [51] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2026. Progent: Securing AI Agents with Privilege Control. *arXiv:2504.11703* [cs.CR] <https://arxiv.org/abs/2504.11703>
- [52] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. 2025. PromptArmor: Simple yet Effective Prompt Injection Defenses. (2025). *arXiv:2507.15219* [cs.CR] <https://arxiv.org/abs/2507.15219>
- [53] Shoaib Ahmed Siddiqui, Radhika Gaonkar, Boris Köpf, David Krueger, Andrew Paverd, Ahmed Salem, Shruti Tople, Lukas Wutschitz, Menglin Xia, and Santiago Zanella-Béguelin. 2026. Permissive Information-Flow Analysis for Large Language Models. *arXiv:2410.03055* [cs.LG] <https://arxiv.org/abs/2410.03055>
- [54] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2025. SAGA: A Security Architecture for Governing AI Agentic Systems. (2025). *arXiv:2504.21034* [cs.CR] <https://arxiv.org/abs/2504.21034>

- [55] The Linux Kernel documentation. 2024. *Seccomp BPF (SECure COMputing with filters)*. The Linux Kernel documentation. https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html Accessed: 2026-09-15.
- [56] Ken Thompson. 1968. Programming Techniques: Regular expression search algorithm. *Commun. ACM* 11, 6 (1968), 419–422. doi:10.1145/363347.363387
- [57] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems (HotOS '25)*. Association for Computing Machinery, New York, NY, USA, 8–17. doi:10.1145/3713082.3730378
- [58] Valve Software. 2015. *Deleted all my user data*. steam-for-linux issue #3671. <https://github.com/ValveSoftware/steam-for-linux/issues/3671> Accessed: 2026-09-15.
- [59] Kenton Varda, Sunil Pai, and Ketan Gupta. 2026. *Sandboxing AI agents, 100x faster*. The Cloudflare Blog. <https://blog.cloudflare.com/dynamic-workers/>
- [60] Eric Wallace, Christopher A. Choquette-Choo, Nikhil Kandpal, Sam Toyer, Dylan Hunn, Stephanie Lin, Yuxin Wen, Xiangyu Qi, Christopher Wolff, Zizhao Wang, Milad Nasr, Sicheng Zhu, Chuan Guo, Juan Felipe Cerón Uribe, Kaiwen Wang, Aiden Low, Kai Xiao, and Kai Chen. 2026. GPT-Red: Automated Red Teaming via Self-Play at Scale. (2026). arXiv:2607.26115 [cs.CR] <https://arxiv.org/abs/2607.26115>
- [61] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv* (2024). arXiv:2404.13208 [cs.CR] <https://arxiv.org/abs/2404.13208>
- [62] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. (2025). arXiv:2503.18666 [cs.AI] <https://arxiv.org/abs/2503.18666>
- [63] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*.
- [64] Simon Willison. 2022. *Prompt injection attacks against GPT-3*. <https://simonwillison.net/2022/Sep/12/prompt-injection>
- [65] Simon Willison. 2022. *You can't solve AI security problems with more AI*. <https://simonwillison.net/2022/Sep/17/prompt-injection-more-ai/>
- [66] Simon Willison. 2023. *Delimiters won't save you from prompt injection*. <https://simonwillison.net/2023/May/11/delimiters-wont-save-you>
- [67] Simon Willison. 2023. *The Dual LLM pattern for building AI assistants that can resist prompt injection*. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>
- [68] Josh Woodward. 2026. *The Gemini app becomes more agentic, delivering proactive, 24/7 help*. <https://blog.google/innovation-and-ai/products/gemini-app/next-evolution-gemini-app/> Accessed: 2026-09-15.
- [69] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. 2024. System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective. arXiv:2409.19091 [cs.CR] <https://arxiv.org/abs/2409.19091>
- [70] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. <https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/>
- [71] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. Benchmarking and defending against indirect prompt injection attacks on large language models. arXiv:2312.14197
- [72] Ren Yi, Octavian Suci, Adrian Gascon, Sarah Meiklejohn, Eugene Bagdasarian, and Marco Gruteser. 2025. Privacy Reasoning in Ambiguous Contexts. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=0ZnXGzLcOg>
- [73] Peter Yong Zhong, Siyuan Chen, Ruiqi Wang, McKenna McCall, Ben L. Titzer, Heather Miller, and Phillip B. Gibbons. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. arXiv:2502.08966 [cs.CR] <https://arxiv.org/abs/2502.08966>